Final Report: Safety Verification of Chiron IR using Constrained Horn Clauses

Project Overview

Program verification provides mathematical guarantees that software behaves correctly, unlike testing, which can find bugs but never prove their absence. While the Chiron framework offers support for dynamic analysis techniques such as fuzzing, symbolic execution, and spectrum-based fault localization, it currently lacks the capability to *prove* that a program satisfies a given specification for *all* possible inputs.

This project bridges the gap between testing and verification in Chiron by implementing a PDR-based safety verification engine. We target properties natural to turtle graphics programs, such as spatial bounds on the turtle's position and invariants over program variables.

Team Information

Roll No.	**Roll No.**	**Email ID**
Aditi Khandelia	220061	aditikh22@iitk.ac.in
Arush Upadhyaya	220213	arushu22@iitk.ac.in
Kushagra Srivastava	220573	skushagra22@iitk.ac.in

Implementation

Summary of Implementation Progress

The verifier now supports a complete CHC-based verification flow for Chiron programs: parsing source into IR, extracting user variables and loop counters, constructing symbolic state relations, generating instruction-level transition rules, and checking user properties through Z3 Fixedpoint/SPACER. The core state model includes program counter, turtle coordinates, heading, pen state, user variables, and internal counters, and the tool reports clear verdicts (`PASSED`, `FAILED`, `UNKNOWN`) using a structured return object with timing and per-property outcomes. The verification interface is fully callable through `CHC_Verification(...)` and supports robust input validation, controlled property parsing, and consistent result reporting.

In addition to baseline verification, the implementation includes explicit heading-grid safety checking through a dedicated `BadHeading` relation, configurable scope controls (`all`, `terminating`, `at_pc`, `upto_pc`), and user-selectable hint behavior for performance-vs-assumption tradeoffs. A basic optimization pipeline has been integrated with static turn-safety analysis, repeat-loop summarization, and exact 15-degree trigonometric handling on the heading lattice. To improve usability, a separate semantic helper transpiler was added so readable user-facing helper expressions can be lowered to solver-ready property strings without modifying the core verifier. The implementation is supported by expanded correctness and performance test coverage across modes, scopes, heading behavior, and optimization settings.

Verification Modes and Property Scopes

Verification Modes

1. **default.** Verifies from a single concrete start state: `pc=0`, `xcor=0`, `ycor=0`, `heading=0`, `pendown=False`, all user variables initialized to `0`, and all loop counters initialized to `0`. This is the baseline configuration for proving invariants from Chiron's standard start state.
2. **specific.** Verifies from a concrete parameterized start state. The caller supplies values for user variables through `params`; any omitted user variable is defaulted to `0`. The rest of the state remains fixed at the standard start configuration (`pc=0`, zero position, zero heading, pen up, counters zero).
3. **universal.** Verifies over a symbolic family of initial states. The verifier fixes `pc=0`, `pendown=True`, and all counters to `0`; constrains `heading` to the 15-degree grid; and leaves `xcor`, `ycor`, and user variables unconstrained. Optional `input_ranges` can additionally restrict selected user variables.

Property Scopes

1. **all.** Checks whether the property holds for every reachable state in the verifier's symbolic model, regardless of control location.
2. **terminating.** Restricts checking to reachable terminal states, i.e. states whose `pc` equals the terminal program counter. If the verifier cannot establish reachability of termination, the result is conservatively reported as `UNKNOWN`.
3. **at_pc.** Restricts checking to reachable states at one exact control point `pc_target`. This scope requires an explicit `pc_target`; if that program counter is unreachable, the scoped query is returned as `UNKNOWN`.
4. **upto_pc.** Restricts checking to the reachable execution prefix satisfying `0 <= pc <= pc_target`. This is useful for prefix invariants that should hold before or up to a particular program point.

The API also enforces a few scope-specific restrictions: `pc_target` is required for `at_pc` and `upto_pc`, forbidden for `all` and `terminating`, and the pc-scoped options are disabled when `optimization_level=BASIC` because loop summarization currently targets whole-program or terminal-state reasoning.

Optimization Levels

1. **OptimizationLevel.NONE.** Uses the baseline encoding: the verifier emits the standard per-instruction CHC rules and supports all four property scopes (`all`, `terminating`, `at_pc`, `upto_pc`).
2. **OptimizationLevel.BASIC.** Enables the current optimization pipeline, namely turn-safety suppression of unnecessary `BadHeading` rules, repeat-loop summarization, and conditional branch merging inside summarized loop bodies. This level is intended for whole-program or terminal-state reasoning, so the API rejects `at_pc` and `upto_pc` queries when `BASIC` is selected.

End-to-End Pipeline View

Figure 1 shows the full safety-verification flow from the Chiron CLI and Python entrypoints into the core CHC engine, including fixedpoint construction, optimization-aware rule emission, semantic gates, and structured result reporting.

File-wise Implementation Details

1. `CHC-Verification/variable_name_detection_in_IR.py` Extracts user variables and internal repeat counters from linear IR. Also defines optimization-level controls that are consumed during CHC generation. Variable extraction uses recursive AST traversal through `parse_variables_from_ir_expr()` and `parse_variables_from_ir_cond()`, producing a `symbol_table` of user-declared variables and a `counter_table` of loop iteration variables. These two tables determine the arity and argument ordering of the `Inv` relation declared in the next stage, so the accuracy of extraction directly governs the completeness of the symbolic state model.

2. `CHC-Verification/z3_fixed_point.py` Builds the mode-aware state signature and declares core relations such as `Inv` and `BadHeading`. This module defines the relation arity and state-vector shape used by downstream rules. The full state vector carries program counter, turtle coordinates (`xcor`, `ycor`), heading, pen state, all

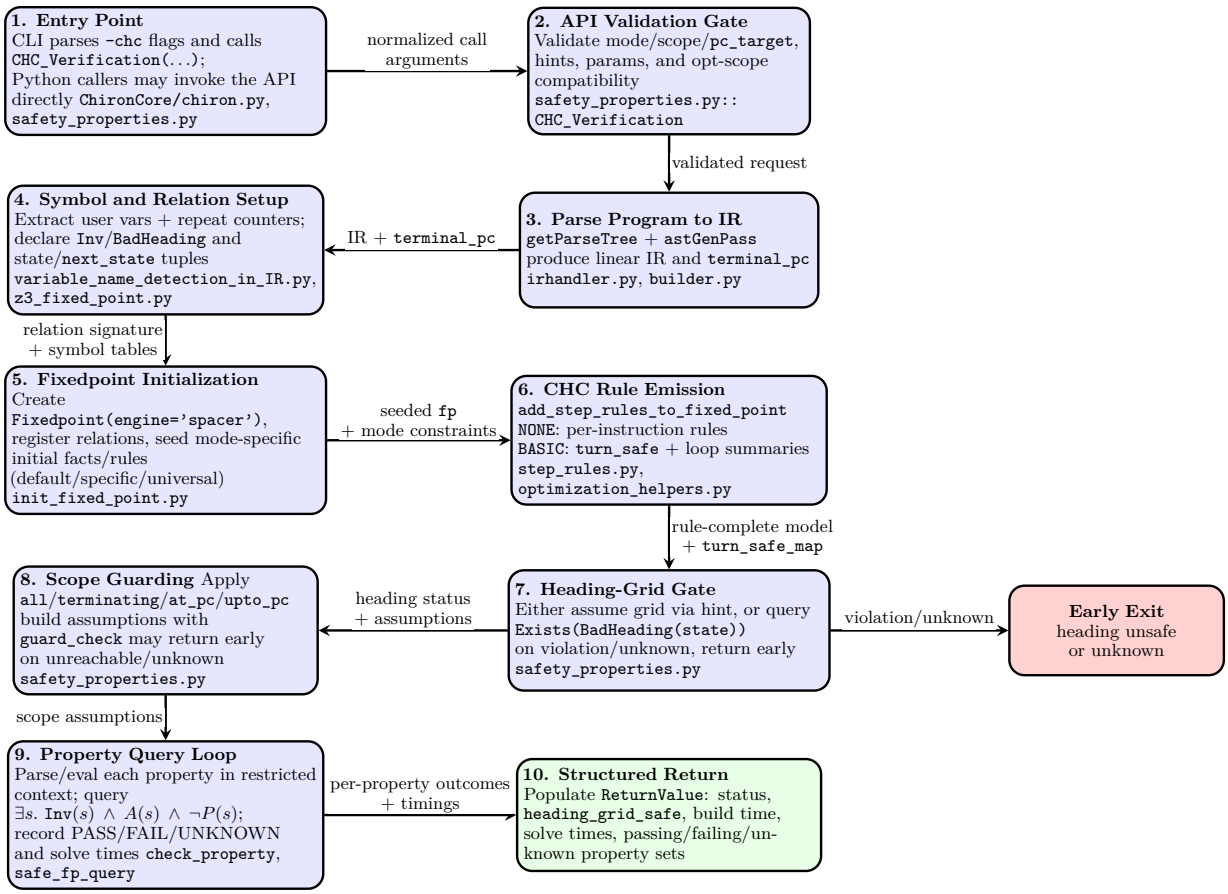

Figure 1: End-to-end safety verification pipeline from CLI/API entry to CHC construction, solver querying, and result delivery. Each stage is annotated with its primary file-level owner.

user variables, and all loop counters; both a *current-state* and a *next-state* copy are constructed so transition rules can be expressed as implications between them. `BadHeading` shares the same state-vector shape as `Inv`, allowing heading-violation rules to reuse the same variable symbols and be registered in the same fixedpoint context without duplicating infrastructure.

3. `CHC-Verification/init_fixed_point.py` Initializes the Z3 Fixedpoint context (SPACER engine), registers relations, and seeds base constraints according to verification mode (`default`, `specific`, `universal`). In `default` mode the initial fact pins all state components to concrete values (`pc=0`, coordinates zero, heading zero, pen up, all variables zero); in `universal` mode coordinates and user variables are left unconstrained while heading is restricted to the 15-degree grid and pen is forced down, capturing the widest possible input class. The `specific` mode accepts a caller-supplied parameter dictionary, initializes the corresponding user variables to those concrete values, defaults omitted user variables to zero, and keeps the remaining state at the standard start configuration. This enables targeted property checking from a concrete parameterized initial state. The same module also normalizes and validates `input_ranges` for universal mode. A variable may be given as a fixed integer, a closed integer interval `(lo, hi)`, or `None` to leave it unconstrained; unknown variable names, non-integer bounds, and malformed ranges are rejected eagerly before any CHC rules are emitted.

4. `CHC-Verification/step_rules.py` Translates each IR instruction into CHC transition rules. It covers assignments, conditionals, movement, heading updates, pen-state updates, assertions, and control-flow jumps. This module is also where optimization-aware rule paths and loop summarization hooks are integrated. For movement instructions, the rule encodes exact integer arithmetic for heading multiples of 15 degrees by consulting the precomputed trigonometric table in `optimization_helpers.py`, avoiding floating-point imprecision in the symbolic encoding. In `BASIC` optimization mode, loops that pass the summarizability check are replaced by a single aggregate transition via `summarize_loop_effect()`, reducing the clause count

submitted to SPACER and improving convergence on loop-heavy programs while leaving non-summarizable fragments on the standard per-instruction path. Assertions are encoded as hard safety obligations rather than advisory checks: for an `AssertCommand`, the true branch advances normally, while the false branch implies `False` in the fixedpoint system. This turns assertion failure directly into reachability of an inconsistent state, so violated assertions are handled by the same CHC safety machinery as user-supplied properties.

5. `CHC-Verification/safety_properties.py` Implements the top-level API `CHC_Verification(...)`. It validates inputs, parses hints, applies scope guards, checks heading-grid safety, evaluates user properties, and returns structured outcomes. Each user property is evaluated inside an `eval()` context populated with the current Z3 state variables, so property strings can reference program variables by name without requiring the caller to import Z3 directly. The return object `ReturnValue` bundles an overall verdict (`SUCCESS`, `ERROR`, or `UNKNOWN`), a separate `heading_grid_safe` field, wall-clock build and solve timings, and three property lists (`passing`, `failing`, `unknown`). This module also contains a small robustness layer around SPACER queries. The helper `safe_fp_query()` retries a query with `spacer.native_mbp=False` when Z3 raises the known `mbp to-real` exception on mixed Int/Real reasoning, and `guard_check()` proactively disables native MBP before reachability checks. This does not change the logical encoding, but it makes the verifier materially more stable on geometry-heavy benchmarks.

6. `CHC-Verification/heading_grid.py` Defines heading-lattice predicates used to prove or assume 15-degree grid safety. The core function `heading_on_grid()` generates a Z3 disjunction over all 24 valid headings $(0°, 15°, \ldots, 345°)$, which is embedded directly as a guard in movement and turn transition rules rather than checked separately. This predicate is also the conclusion of the `BadHeading` violation rules: any transition that produces a heading outside the grid implies `BadHeading` on the resulting state, allowing SPACER to derive a reachability witness if the invariant is broken.

7. `CHC-Verification/optimization_helpers.py` Implements static turn-safety reasoning, repeat-loop detection, summarizability checks, and exact 15-degree trig tables used by optimized rule generation. The precomputed `_MULT15_VALUES` dictionary maps each of the 24 valid headings to decimals Δx and Δy displacements scaled, so movement rules submitted to SPACER contain only integer arithmetic other than *xcor* and *ycor* update. The `is_summarizable_loop_nested()` predicate enforces structural constraints - a positive constant loop bound within `MAX_SUMMARIZE_ITERATIONS`, no `AssertCommand` nodes, only unit jumps except for supported if/else shapes, and no use of the loop counter inside the body (already summarized inner loops are treated as atomic) - so that the summarization in `step_rules.py` is provably sound: every concrete execution through the loop is covered by the emitted summary transition.

8. `CHC-Verification/semantic_properties.py` Provides a user-facing helper-string transpilation layer that lowers readable semantic helpers into raw CHC-compatible property expressions without modifying the core verifier. Helper functions such as `is_nonnegative(v)`, `position_in_box(...)`, `pen_is_up()`, and `relation_guarded(...)` are resolved before the property string reaches `eval()`, so the caller can write human-readable specifications that are automatically converted to Z3 Boolean expressions over state variables. The companion entry point `CHC_Verification_semantic()` wraps `CHC_Verification()` with this transpilation step, keeping the semantic helper layer fully optional and backward-compatible with callers that already supply raw Z3 expressions.

Design Choices

1. **CHCs solved by Z3 SPACER.** The verifier encodes program behavior as Constrained Horn Clauses and submits them to Z3's `Fixedpoint` with `engine='spacer'`, an IC3/PDR-style Horn-clause solver that performs inductive invariant synthesis.
   - *Benefit:* Supports unbounded safety checking; returns a concrete counterexample witness on violation, not just a yes/no answer.
   - *Tradeoff:* Performance is sensitive to nonlinear arithmetic obligations; complex programs or unconstrained inputs can produce `UNKNOWN` within the timeout.
2. **Single global invariant relation `Inv(pc, xcor, ycor, heading, pendown, vars, counters)`.** Full control-and-data state is captured in one unified relation shared across initialization, all transition rules, and every property query.
   - *Benefit:* `pc` governs control-flow and termination, geometric variables support spatial predicates, and loop counters make repeat semantics explicit without unrolling; all pipeline stages have this signature.
   - *Tradeoff:* Properties over a strict variable subset still incur full-arity SPACER reasoning; adding any new state component requires regenerating the entire relation signature and all registered rules.

3. **Strict heading lattice with explicit `BadHeading` relation.** Heading is restricted to $\{0°, 15°, \ldots, 345°\}$ and movement arithmetic is handled via a precomputed 24-entry fractional lookup table rather than transcendental `sin`/`cos` constraints.
   - *Benefit:* Keeps all arithmetic linear, with only `xcor` and `ycor` as decimals, others as integer, which SPACER handles reliably; heading violations are a first-class proof obligation rather than a silent assumption.
   - *Tradeoff:* The disjunctive heading guard expands CHC clause size proportionally to the number of valid headings; failing to prove grid preservation yields a conservative `UNKNOWN` on all downstream property checks.

Optimisations

Files: CHC-Verification/optimization_helpers.py, CHC-Verification/step_rules.py

The following optimisations are implemented at `OptimizationLevel.BASIC` to reduce the number and complexity of CHC rules submitted to SPACER, improving performance on loop-heavy programs and those with many turn instructions while preserving soundness.

Heading Check Optimisation

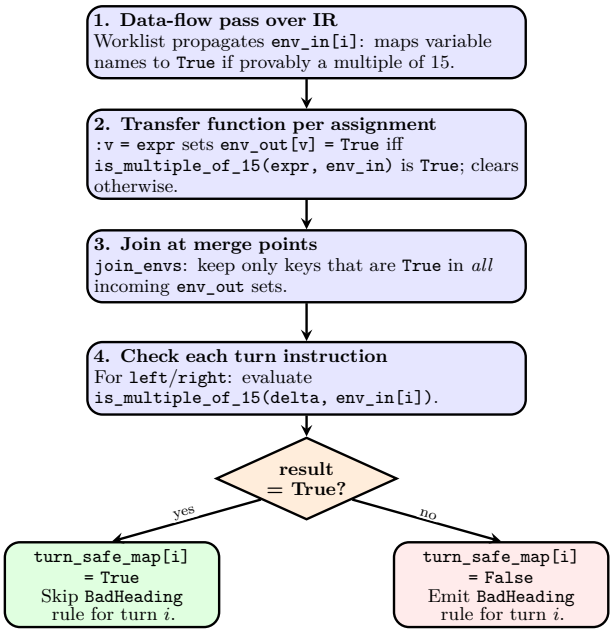

Figure 2: Turn-safe heading inference: data-flow analysis that marks turn instructions whose delta is statically provable as a multiple of 15.

Each turn instruction (`left`/`right`) in the standard encoding generates a `BadHeading` CHC rule asserting that the resulting heading might leave the grid. The heading-check optimisation eliminates these rules wherever the turn amount is statically proven to be a multiple of 15.

1. A forward data-flow analysis (`compute_env_in`) propagates, for each program point, the set of variables known to hold a multiple-of-15 value. The transfer function `transfer_env` updates this set on assignments: if the right-hand side evaluates to `True` under `is_multiple_of_15`, the left-hand variable is added; otherwise it is removed.
2. At merge points (after conditionals), `join_envs` takes the intersection — a variable is kept only if it is provably a multiple of 15 on *every* incoming path, preserving soundness across branches.
3. `is_multiple_of_15` handles numeric literals (`n % 15 == 0`), variable lookups, unary minus, sums and differences (both operands must be known), and multiplication (either factor must be known). Any unsupported shape returns `None` (unknown), triggering conservative `BadHeading` emission.

4. If a turn's delta resolves to `True` at its program point, `turn_safe_map[i]` is set and the `BadHeading` rule for that instruction is suppressed, reducing the clause count submitted to SPACER.

Loop Optimisation

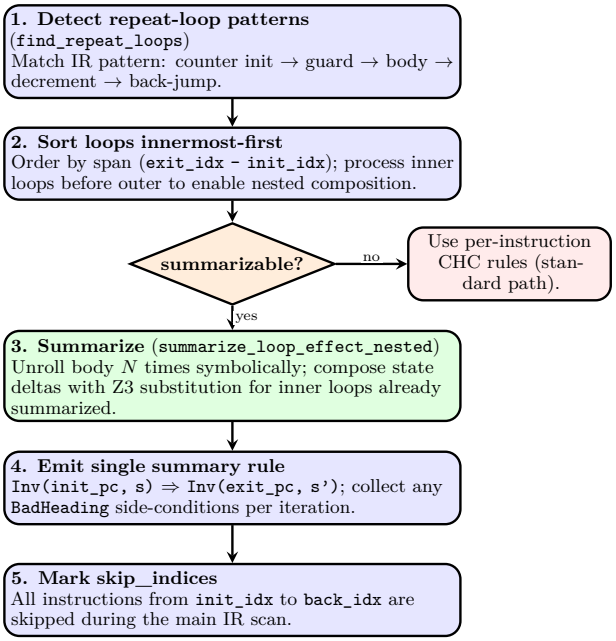

Figure 3: Repeat-loop summarization: innermost-first pattern matching, summarizability check, symbolic unrolling, and single-rule emission.

Instead of emitting one CHC rule per instruction for every iteration of a `repeat` loop, the loop optimisation compiles the entire loop into a single transition from loop entry to loop exit.

1. `find_repeat_loops` scans the IR for the canonical 6-instruction repeat pattern: counter initialisation, a `counter > 0` guard, a loop body, a `counter = counter - 1` decrement, and a back-jump. This pattern is produced deterministically by Chiron's parser.
2. `is_summarizable_loop_nested` checks structural preconditions: the loop count must be a positive integer not exceeding `MAX_SUMMARIZE_ITERATIONS` (150), the body must contain no `AssertCommand` nodes, and every non-conditional instruction must have unit jump offset. Already-summarized inner loops are treated as opaque atomic blocks during this check.
3. Loops are processed innermost-first. `summarize_loop_effect_nested` unrolls the body N times symbolically, carrying a running Z3 state expression rather than adding solver variables. When an inner loop is encountered, its pre-computed state transformation is applied via Z3 `substitute` rather than stepping through its instructions again.
4. The result is one `fp.rule(ForAll(vars, Inv(init_pc, s) ⇒ Inv(exit_pc, s')))` rule replacing the entire loop, plus any `BadHeading` side-condition rules accumulated during unrolling. All instructions within the summarized range are added to `skip_indices` and bypassed in the main IR scan.

Conditional Optimisation

When a loop body contains if/else branches, the summarizer must represent both paths as a single symbolic state expression rather than splitting into multiple CHC rules.

1. `_execute_segment_with_branch_merge` handles two conditional shapes produced by Chiron's parser: a `BoolFalse` skip-goto (used to jump over the else block at the end of the then-branch), and a full structured if/else with a true-branch terminator goto followed by the else block.
2. If the branch condition is statically `True` or `False` at summarization time (Z3's `is_true`/`is_false`), the dead branch is dropped entirely and no `If`-expression is introduced, keeping the resulting CHC rule compact.
3. For the general case, both branches are executed symbolically under their respective path guards (`And(pg, cond)` and `And(pg, Not(cond))`), and each state component is merged independently using `_merge_branch_value`.

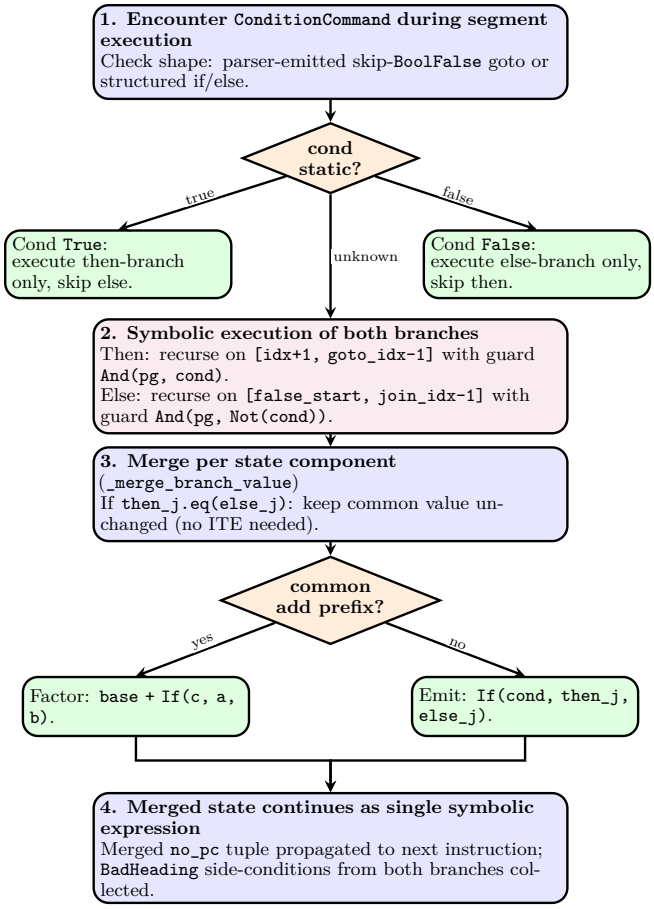

Figure 4: Conditional handling inside summarized loop bodies: static branch elimination and ITE merging with common-prefix factoring.

4. `_merge_branch_value` first checks if then and else values are syntactically equal (`then_j.eq(else_j)`); if so, the common value is kept unchanged without any `If`. Otherwise it looks for a common additive prefix: `If(c, base+a, base+b)` is rewritten to `base + If(c, a, b)`, reducing expression depth in branch-heavy loops. If neither shortcut applies, a plain `If(cond, then_j, else_j)` is emitted.
5. `BadHeading` side-conditions accumulated in both branches are collected and emitted together with the summary rule, so heading-grid obligations inside conditional bodies are not lost.

Hints

File: CHC-Verification/safety_properties.py

Hints are optional caller-supplied strings passed via the `hints` parameter that let the user trade a proof obligation for an assumption, skipping a solver query when the property is already known to hold. All hints are validated at API entry against the `Hints` enum; an unrecognised value is rejected before any solving begins.

- **`check_heading_always_on_grid`** (default). Proves that the turtle's heading stays on the 15-degree grid by querying `BadHeading`. The result is stored in `ReturnValue.heading_grid_safe`; if the check fails or is unknown, the verifier stops before evaluating user properties.
- **`heading_on_grid_always`**. Assumes the heading invariant rather than proving it. The `BadHeading` query is skipped and `heading_on_grid(state[3])` is added directly as an assumption. Useful when all turns are provably multiples of 15 by construction, saving one SPACER call.

- `check_termination` (default). When `property_scope="terminating"`, first queries whether the terminal PC is reachable before checking properties there. If SPACER cannot confirm termination, the run returns `UNKNOWN` without attempting the property checks.
- `always_terminates`. Skips the termination reachability check and assumes the terminal state is reachable, adding the terminal-PC guard directly as an assumption. Appropriate for programs with only finite `repeat` loops and no infinite branches.

The two heading hints and the two termination hints form mutually exclusive pairs: supplying both members of a pair is rejected at validation since one proves and the other assumes the same property.

Tests

Correctness Tests

The correctness suite resides in *Folder: CHC-Verification/correctness_tests* and is executed with `pytest`, using `pytest-xdist` for parallel execution, over `unittest.TestCase`-based test classes. The fixture set contains **64** `.tl` programs shared across all test modules, covering a wide range of semantic properties and edge cases. These fixture programs can be found in *Folder: CHC-Verification/correctness_tests/programs/*.

Test Modules

File	Role in evaluation
`test_default.py`	Validates concrete-start-state verification in `default` mode, covering arithmetic invariants, geometric bounds, pen state, directional and heading-grid checks, trigonometric motion, and advanced nested control-flow fixtures.
`test_specific.py`	Validates parameterised initialisation in `specific` mode, with boundary-sensitive outcomes under user-supplied parameter dictionaries; exercises countdown, accumulate, chain, two-param, loop-move, nested conditional, and pen-conditional programs.
`test_universal.py`	Validates symbolic/unconstrained initialisation in `universal` mode, API-level validation errors, heading-grid safety queries, pc-scoped (`at_pc`/ `upto_pc`) and terminating-scope properties across arithmetic, geometric, pen, and heading categories.
`test_semantics.py`	Validates the semantic-helper transpiler (`semantic_properties.py`) at two levels: (i) unit tests that assert the `transpile_property_expr` function converts every built-in helper call to the correct Z3 expression string; (ii) end-to-end integration tests that invoke `CHC_Verification` with helper-style property strings across all three modes (default, specific, universal).

Fixture Programmes

The 64 fixture programmes are organised into ten categories below.

Programme file	Purpose in evaluation
Linear Arithmetic (Assignment / Loop / Conditional)	
`assign_basic.tl`	Simple assignment sequence: x=10, z=30; baseline for arithmetic invariants.
`assign_algebra.tl`	Chained algebraic assignments: a=5, b=a+30, c=b×5.
`conditional.tl`	Single if-else; assigns z depending on the branch taken.
`loop_basic.tl`	Three-step loop incrementing x by a fixed amount each iteration.
`loop_accum.tl`	Accumulator: x grows by 10 per step over 5 iterations.
`loop_adv.tl`	Advanced accumulator; x follows a triangular-number growth sequence.
`loop_cond.tl`	Loop with conditional body; y decrements only while $x <$ threshold.
`loop_nested.tl`	Two-level nested loop; inner counter depends on outer-loop state.
`loop_xy_dep.tl`	Loop with interdependent x and y updates.
`read_before_write.tl`	Reads `:step` before any write; $z = y +$ step at termination.

Programme file	Purpose in evaluation
Goto / Geometric	
`square_goto.tl`	Draws a 50×50 square using `goto`; xcor, ycor trace four corners.
`goto_computed.tl`	`goto` with computed coordinates; `xcor=40`, `ycor=20` at end.
`goto_accum.tl`	Accumulates goto positions; `xcor` grows by fixed steps.
`forward_square.tl`	`forward`/`right 90` loop; heading cycles through quarter-turns only.
`spiral_grid_goto.tl`	Spiral-like goto sequence; coordinates increase monotonically.
Pen State	
`pen_only.tl`	Single `pendown` command; no arithmetic changes.
`pen_toggle.tl`	Alternates `pendown`/`penup` on successive iterations.
`pen_with_var.tl`	`pendown` executed after a variable assignment.
Turns / Heading	
`turns_only.tl`	Three `right` commands; heading cycles through {0, 270, 180, 90}.
`turns_15.tl`	Repeated `right(15)` turns; heading stays on 15-degree grid.
`branchy_non_grid.tl`	Branching with a non-15-degree turn; heading leaves the 15-degree grid.
`maze_counter.tl`	Counter-driven path; heading changes interleaved with `forward`.
Nested Loops / Advanced Control Flow	
`flower_nested_pen.tl`	Nested-loop motif with `pendown`; outer counter `count` tracked.
`triangle_nested_pen.tl`	Two-level nested loop for a triangle; `step` and `side` counters.
Parameterised (for Specific Mode)	
`param_cond.tl`	Conditional branch driven by `:init`; outcome depends on value relative to 50.
`param_countdown.tl`	Counter counts down from `:n` over 5 steps; boundary at n=5.
`param_accumulate.tl`	$total = base + k \cdot inc$ for $k = 0 \ldots 3$; two-param boundary.
`param_chain.tl`	Chain: a=*seed*, b=a+1, c=a+b, d=b+c; boundary at *seed*=6.
`param_goto.tl`	`goto(:startx, :starty)`; position driven entirely by params.
`param_loop.tl`	`acc` starts at `:start` and increments three times.
`param_loop_move.tl`	x=*start*, x+=*step* for 3 iters, `goto`$(x, 0)$ each iteration.
`param_nested_cond.tl`	Nested conditional with two boundary points at 0 and 100.
`param_pen.tl`	*mark*=*base*+1; `pendown` always executes.
`param_pen_cond.tl`	`pendown` fires only when `:value` > `:threshold`.
`param_scale.tl`	*result* = *scale* × 5; single-param arithmetic.
`two_params.tl`	*diff*=*hi*−*lo*, *sum*=*hi*+*lo*; relational outcome depends on both.
Arithmetic (for Universal Mode)	
`u_abs_diff_branching.tl`	Branching produces $\|x - y\|$; two-branch affine summary.
`u_branch_merge_affine.tl`	Branch merge: d=$\|x-y\|$, s=x at termination.
`u_clamp_to_zero.tl`	$z = \max(x, 0)$; disjunctive postcondition z=0 ∨ z=x.
`u_euclid_avg.tl`	a=m+n, b=m−n, c=a+b=2m; arithmetic identity chain.
`u_product_zero.tl`	z=0 regardless of x, y; one factor is forced to zero.
`u_same_postcondition.tl`	Both branches produce z=0; identical postcondition across paths.
`u_swap_with_temp.tl`	Swap x, y via `tmp`; y=*tmp*, x≠*tmp* at termination.
`u_swap_without_tmp.tl`	In-place swap; s=x+y preserved throughout all states.
`u_two_step_affine.tl`	Two sequential affine updates; linear postconditions expected to fail.
Geometric / Position (for Universal Mode)	
`u_fixed_target.tl`	`goto(10, 20)`; exact position at termination.
`u_fixed_target_user.tl`	`goto(a, a+7)`; *ycor*−*xcor*=7 at termination.
`u_normalize_x.tl`	`goto(5, 5)`; `xcor=ycor=5` regardless of starting position.
`u_loop_diagonal.tl`	Loop with `forward` and heading changes; exercises heading grid inside loops.
`u_xcor_fixed.tl`	`xcor=8`; `ycor=3` or `ycor=-3` depending on sign(p).
Pen State (for Universal Mode)	
`u_branch_pen.tl`	`pendown` in else-branch (x<0); pen state correlated with guard.
`u_branch_pen_adv.tl`	Advanced pen-branch: condition a+b<b simplifies to a<0.
`u_loop_pen.tl`	`pendown` inside a loop; pen ends down at termination.
`u_pen_no_touch.tl`	No `penup` in program; `pendown=True` holds throughout.
`u_pen_up.tl`	`penup` at end; `pendown=False` holds at termination.

Programme file	Purpose in evaluation
Heading / Turns (for Universal Mode)	
`u_branch_always_mul_15.tl`	Branch assigns turn of 30 or 45 (both multiples of 15).
`u_branch_always_mul_15_adv.tl`	Advanced version: t reduces to 30 via branch; still a multiple of 15.
`u_fixed_left.tl`	Turns left by a multiple-of-15 angle; heading stays on 15-degree grid.
`u_heading_non_grid.tl`	Turns by a non-multiple-of-15; heading leaves the 15-degree grid.
`u_init_core.tl`	Universal-init skeleton; exposes all universal-mode initial conditions.
`u_mul_15_heading_on_grid.tl`	$t=15\times m$; `left t`; heading stays on 15-degree grid.
`u_net_heading_preserved.tl`	`left a`; `right a`; heading returns to starting value.
`u_net_zero_turn.tl`	`left(45)`; `right(45)`; heading returns to original value.
`u_on_spot_turn.tl`	Turn in place; position unchanged, heading changes.

Correctness Results

Table 3 summarises the outcome of running all four test modules against under a 20 s solver timeout. Pass rate is computed over the tests that did not time out.

Table 3: Correctness test outcomes within the 20 s solver timeout (all active tests pass with 0 failures/timeouts).

Module	Total	Passed	Failed	Timeout	Pass Rate
`test_default.py`	46	45	0	1	100%
`test_specific.py`	70	70	0	0	100%
`test_universal.py`	112	112	0	0	100%
`test_semantics.py`	130	130	0	0	100%
Total	**358**	**357**	**0**	**1**	**100%**

Crucially, the timeouts reflect a *solver scalability* limitation, not a defect in the encoding logic. The affected programs are correctly translated into CHC rules, and the properties are well-formed; Spacer simply cannot synthesize a sufficiently strong inductive invariant within the configured timeout when both heading normalization and trigonometric approximations are present in the same fixed-point system.

How to Run

The correctness suite is driven by `pytest` from the `CHC-Verification/correctness_tests/` directory. Parallel execution via `pytest-xdist` (`-n auto`) is recommended.

```
# runs all tests in parallel
pytest -n auto -q

# one module only
pytest -n auto -q <test_module.py>

# one test case
pytest test_semantics.py::TestSemanticPenUniversal::
    test_branch_pen_terminating_guarded_relation_pass -q
```

Performance Tests

The CHC verification pipeline is benchmarked to stress-test solver scalability under various optimisation levels and hint configurations. The test suite resides in *Folder: `CHC-Verification/performance_tests`* and is executed with `pytest` using `pytest-xdist` for parallel execution.

The fixture contains 24 `.tl` programs across the three modes and cover cases that can blow up the number of solver constraints. The fixture programmes can be found in *Folder: `CHC-Verification/performance_tests/programs/`*. Each test case is evaluated under **six configurations** formed by crossing two optimisation levels with three hint modes (detailed in the *Hint Configurations* subsection). Timing data are written to `perf_results_{default,universal,specific}.csv`.

Test Modules

File	Purpose
`test_default.py`	Performance benchmarks in `default` mode, covering loop scaling, nesting depth, wide state, branching density, trigonometric motion, heading-grid cost, and property complexity.
`test_universal.py`	Performance benchmarks in `universal` mode with unconstrained initial states; mirrors the default-mode categories but adds `all`/`terminating` scope variants.
`test_specific.py`	Performance benchmarks in `specific` mode using explicit parameter assignments; includes offset-start variants and specific-mode property-complexity tests.

Fixture Programmes

The performance suite uses **24** benchmark programmes organised into six categories.

Programme file	Purpose in evaluation
Loop Scaling	
`perf_repeat_10.tl`	`:x = 0; repeat 10 [:x = :x + 1]`; x=10 at termination.
`perf_repeat_50.tl`	Same body as `perf_repeat_10.tl`, 50 iterations; x=50 at termination.
`perf_repeat_100.tl`	Same body as `perf_repeat_10.tl`, 100 iterations; x=100 at termination.
`perf_repeat_5000.tl`	Same body as `perf_repeat_10.tl`, 5000 iterations; x=5000 at termination.
`perf_s_repeat_50.tl`	`:x = :init; repeat 50 [:x = :x + 1]`; x=*init*+50 at termination.
`perf_s_repeat_100.tl`	Same body as `perf_s_repeat_50.tl`, 100 iterations; x=*init*+100 at termination.
`opt_found.tl`	`:a = 0; :b = 0; repeat 100 [:a = :a + 1; :b = :b + :a]`; triangular accumulator, b=$a(a+1)/2$.
`opt_universal_100.tl`	Same body as `opt_found.tl` but a not initialised; b+=a loop with symbolic start.
Nesting Depth	
`perf_deep_nest_3.tl`	Three-level `repeat 4` loops; a, b, c each incremented at their level; a=4, b=16, c=64 at termination.
`perf_deep_nest_4.tl`	Four-level `repeat 3` loops; a, b, c, d incremented at their level; a=3, b=9, c=27, d=81 at termination.
`perf_s_nest_3.tl`	Three-level `repeat 5` loops; a, b, c initialised to `:start`; a=*start*+5, b=*start*+25, c=*start*+125.
`perf_s_nest_4.tl`	Four-level `repeat 4` loops; a, b, c, d initialised to `:start`; tight bound on d shifts by `:start`.
Wide State	
`perf_wide_vars.tl`	Eight variables a=1, …, h=8; 5-iteration loop with b+=a, c+=b, …, h+=g; 8-dimensional affine state.
`perf_s_wide.tl`	Same chained-update body as `perf_wide_vars.tl` with all initial values from params; 8-dimensional symbolic state.
Branching Density	
`perf_many_branches.tl`	6-iteration loop; x+=1 each iter drives four sequential guards (x>1, x>2, x>3, x>4), each incrementing *acc*.
`perf_s_branches.tl`	8-iteration loop; same four-guard structure as `perf_many_branches.tl` but thresholds t_1=*thresh*, t_2=*thresh*+1, t_3=*thresh*+2, t_4=*thresh*+3.
`perf_combo.tl`	5-iteration loop; *step*+=1 each iter; if *step*>2: `forward step; right 90; acc += step`, else: `left 90; acc += 1`; combines branching and motion.
Trigonometric Motion	
`perf_trig_10.tl`	`repeat 10 [forward 10; right 90]`; traces a square, returning to origin with heading 0.
`perf_trig_20.tl`	Same body as `perf_trig_10.tl`, 20 iterations; two full square traces.

Programme file	Purpose in evaluation
`perf_s_trig_10.tl`	`repeat 10 [forward :step; right 90]`; same square pattern with symbolic step magnitude.
`perf_s_trig_20.tl`	Same body as `perf_s_trig_10.tl`, 20 iterations.
Heading / Turns	
`perf_turns_20.tl`	`repeat 20 [right 15; left 15]`; net-zero turn each iteration; heading returns to initial value.
`perf_turns_50.tl`	Same body as `perf_turns_20.tl`, 50 iterations.
`turn_mul_sum.tl`	`:k = 15 * (:x + :y); right :k`; single turn by symbolic multiple of 15; heading-grid membership with unconstrained x, y.

Performance Results

Each benchmark test is run under **six configurations**:

Label	Optimisation	Hint
NONE	`NONE`	(none)
NONE+H1	`NONE`	`check_heading_always_on_grid`
NONE+H2	`NONE`	`heading_on_grid_always`
BASIC	`BASIC`	(none)
BASIC+H1	`BASIC`	`check_heading_always_on_grid`
BASIC+H2	`BASIC`	`heading_on_grid_always`

Default Mode

Each row shows the *best* configuration (lowest total time) for that property. Speedup is computed as Total(NONE)/Total(best); "–" means NONE also timed out.

Table 6: Solver timing benchmark: Default mode.

Program	Config	Property	Verdict	Result	Build (s)	Solve (s)	Total (s)	Speedup
Default mode – Loop Scaling								
`repeat_10`	NONE+H1	$x \geq 0$	`TRUE`	`PASS`	0.026	0.006	0.033	2.6×
`repeat_50`	NONE+H1	$x \geq 0$	`TRUE`	`PASS`	0.029	0.008	0.037	2.2×
`repeat_100`	NONE+H1	$x \geq 0$	`TRUE`	`PASS`	0.025	0.006	0.031	2.6×
`repeat_10`	BASIC+H2	$x \leq 10$	`TRUE`	`PASS`	0.317	0.048	0.365	25.7×
`repeat_50`	BASIC+H2	$x \leq 50$	`TRUE`	`PASS`	0.380	0.070	0.450	44.5×
`repeat_100`	BASIC+H1	$x \leq 100$	`TRUE`	`PASS`	0.350	0.025	0.375	53.5×
`repeat_10`	BASIC+H1	$x \leq 5$	`FALSE`	`PASS`	0.337	0.061	0.398	5.6×
`repeat_50`	BASIC	$x \leq 5$	`FALSE`	`PASS`	0.394	0.059	0.453	4.2×
`repeat_100`	BASIC	$x \leq 5$	`FALSE`	`PASS`	0.341	0.070	0.411	4.4×
Default mode – Nesting Depth								
`deep_nest_3`	NONE+H1	$a, b, c \geq 0$	`TRUE`	`PASS`	0.209	0.021	0.230	1.1×
`deep_nest_4`	NONE+H1	$a, b, c, d \geq 0$	`TRUE`	`PASS`	0.315	0.034	0.349	1.0×
`deep_nest_3`	BASIC+H1	$c \leq 64$	`TRUE`	`PASS`	0.938	0.072	1.010	20.0×
`deep_nest_4`	BASIC+H1	$d \leq 81$	`TRUE`	`PASS`	1.007	0.090	1.098	18.5×
`deep_nest_3`	BASIC	$c \leq 60$	`FALSE`	`PASS`	0.989	0.112	1.101	18.3×
`deep_nest_4`	BASIC	$d \leq 70$	`FALSE`	`PASS`	1.109	0.149	1.258	16.2×
Default mode – Wide State								
`wide_vars`	NONE	$a, \ldots, h > 0$	`FALSE`	`PASS`	0.288	0.072	0.361	1.0×
`wide_vars`	BASIC+H1	$a \leq 6$	`TRUE`	`PASS`	2.966	0.075	3.041	6.7×
`wide_vars`	BASIC	$a \leq 4$	`FALSE`	`PASS`	3.149	0.309	3.457	5.9×
`wide_vars`	BASIC	$h \geq 0$	`TRUE`	`PASS`	2.966	0.466	3.431	5.9×
`wide_vars`	NONE	$h \leq 2211$	-	`TIMEOUT`	0.278	20.024	**T/O**	–
Default mode – Branching Density								
`many_branches`	NONE+H1	$x \geq 0$	`TRUE`	`PASS`	0.190	0.073	0.263	1.0×

Program	Config	Property	Verdict	Result	Build (s)	Solve (s)	Total (s)	Speedup
many_branches	NONE	$acc \geq 0$	TRUE	PASS	0.201	0.080	0.281	1.0×
many_branches	BASIC+H1	$x \leq 6$	TRUE	PASS	0.408	0.096	0.505	40.0×
many_branches	BASIC+H1	$x \leq 3$	FALSE	PASS	0.389	0.090	0.478	42.2×
many_branches	BASIC	$acc \leq 5$	FALSE	PASS	0.387	0.111	0.498	40.6×
Default mode – Trigonometric Motion								
trig_10	NONE+H1	$ycor \geq 0$	FALSE	PASS	0.071	0.388	0.459	1.0×
trig_20	NONE	$ycor \geq 0$	FALSE	PASS	0.105	0.544	0.648	1.0×
trig_10	NONE	$\neg pendown$	TRUE	PASS	0.100	0.079	0.179	1.0×
trig_20	NONE	$\neg pendown$	TRUE	PASS	0.100	0.032	0.132	1.0×
trig_10	NONE+H1	heading $\in \{0, 90, 180, 270\}$	TRUE	PASS	0.105	0.191	0.296	1.0×
trig_20	NONE	heading $\in \{0, 90, 180, 270\}$	TRUE	PASS	0.087	0.082	0.168	1.0×
Default mode – Heading/Turns								
turns_20	NONE	heading $\in \{0, 345\}$	TRUE	PASS	0.092	0.268	0.360	1.0×
turns_50	NONE+H1	heading $\in \{0, 345\}$	TRUE	PASS	0.100	0.264	0.364	1.1×
turns_20	NONE+H1	heading ≥ 0	TRUE	PASS	0.092	0.052	0.144	1.2×
turns_20	NONE+H2	heading on 15° grid	TRUE	PASS	0.094	0.075	0.169	9.4×
Default mode – Property Complexity								
repeat_10	NONE+H1	single atom ($x \geq 0$)	TRUE	PASS	0.037	0.049	0.086	1.1×
repeat_10	BASIC+H2	two-atom conj ($x \geq 0 \wedge x \leq 10$)	TRUE	PASS	0.371	0.077	0.449	42.5×
repeat_10	BASIC+H2	11-way exact disjunction on x	TRUE	PASS	0.377	0.084	0.461	43.5×

Universal Mode

Table 7: Solver timing benchmark: Universal mode.

Program	Config	Property	Verdict	Result	Build (s)	Solve (s)	Total (s)	Speedup
Universal mode – Loop Scaling								
repeat_10	NONE	$x \geq 0$ (terminating)	TRUE	PASS	0.045	0.086	0.131	1.0×
repeat_10	NONE+H1	$x \geq 0$ (all states)	FALSE	PASS	0.040	0.080	0.120	1.0×
repeat_50	NONE+H1	$x \geq 0$	TRUE	PASS	0.043	0.084	0.127	1.0×
repeat_50	NONE+H1	$x \geq 0$ (all states)	FALSE	PASS	0.037	0.064	0.101	1.2×
repeat_100	NONE+H1	$x \geq 0$ (terminating)	TRUE	PASS	0.033	0.076	0.109	1.2×
repeat_100	NONE	$x \geq 0$ (all states)	FALSE	PASS	0.033	0.064	0.098	1.0×
repeat_10	NONE+H2	$x \leq 10$	TRUE	PASS	0.036	0.176	0.211	1.1×
repeat_50	BASIC+H1	$x \leq 50$	TRUE	PASS	0.444	0.058	0.501	3.6×
repeat_100	BASIC	$x \leq 100$	TRUE	PASS	0.372	0.048	0.420	30.4×
repeat_10	NONE	$x \leq 5$	FALSE	PASS	0.032	0.107	0.139	1.0×
repeat_50	BASIC+H1	$x \leq 25$	FALSE	PASS	0.439	0.057	0.496	1.4×
repeat_100	BASIC	$x \leq 50$	FALSE	PASS	0.384	0.049	0.432	46.4×
Universal mode – Nesting Depth								
deep_nest_3	NONE	$c \geq 0$	FALSE	PASS	0.228	0.079	0.307	1.0×
deep_nest_4	NONE	$d \geq 0$	FALSE	PASS	0.389	0.068	0.457	1.0×
deep_nest_3	BASIC	$a, b, c \geq 0$	TRUE	PASS	0.794	0.046	0.840	–
deep_nest_4	BASIC+H2	$a, b, c, d \geq 0$	TRUE	PASS	1.065	0.094	1.159	–
deep_nest_3	BASIC+H2	$c \leq 64$	TRUE	PASS	0.991	0.074	1.065	–

Program	Config	Property	Verdict	Result	Build (s)	Solve (s)	Total (s)	Speedup
deep_nest_4	BASIC+H2	$d \leq 81$	TRUE	PASS	1.084	0.085	1.168	–
deep_nest_3	BASIC+H1	$c \leq 60$	FALSE	PASS	0.859	0.034	0.894	–
deep_nest_4	BASIC+H1	$d \leq 70$	FALSE	PASS	0.597	0.035	0.631	–
Universal mode – Wide State								
wide_vars	NONE+H1	$a, \ldots, h > 0$	FALSE	PASS	0.287	0.071	0.359	1.0×
wide_vars	NONE+H1	$a \leq 6$	TRUE	PASS	0.303	0.143	0.446	1.0×
wide_vars	NONE	$a \leq 4$	FALSE	PASS	0.285	0.171	0.456	1.0×
wide_vars	NONE+H2	$h \geq 0$	TRUE	PASS	0.257	0.512	0.770	1.0×
wide_vars	NONE+H1	$h \leq 2211$	TRUE	PASS	0.298	0.319	0.617	6.8×
wide_vars	NONE+H1	$h \leq 2000$	FALSE	PASS	0.331	0.123	0.454	1.0×
Universal mode – Branching Density								
many_branches	NONE+H1	$x \geq 0$	FALSE	PASS	0.185	0.072	0.258	1.0×
many_branches	BASIC	$acc \geq 0$	TRUE	PASS	0.384	0.042	0.426	–
many_branches	BASIC+H1	$x \leq 6$	TRUE	PASS	0.188	0.043	0.231	55.6×
many_branches	BASIC	$x \leq 3$	FALSE	PASS	0.351	0.031	0.382	–
many_branches	BASIC	$acc \leq 5$	FALSE	PASS	0.351	0.037	0.387	–
Universal mode – Trigonometric Motion								
trig_10	NONE+H1	$ycor \geq 0$	FALSE	PASS	0.103	0.062	0.166	1.1×
trig_10	NONE	heading $\in \{0, 90, 180, 270\}$	FALSE	PASS	0.094	0.055	0.149	1.0×
trig_10	NONE+H1	$0 \leq$ heading $<$ 360	TRUE	PASS	0.094	0.067	0.161	1.0×
trig_10	NONE	$\neg$*pendown*	FALSE	PASS	0.097	0.075	0.172	1.0×
trig_10	NONE+H1	*pendown*	TRUE	PASS	0.077	0.052	0.129	1.4×

Specific Mode

Table 8: Solver timing benchmark: Specific mode.

Program	Config	Property	Verdict	Result	Build (s)	Solve (s)	Total (s)	Speedup
Specific mode – Loop Scaling								
s_repeat_50	NONE	$x \geq 0$	TRUE	PASS	0.046	0.053	0.099	1.0×
s_repeat_100	NONE	$x \geq 0$	TRUE	PASS	0.041	0.078	0.119	1.0×
s_repeat_50	BASIC+H1	$x \leq 50$	TRUE	PASS	0.434	0.084	0.518	38.7×
s_repeat_100	BASIC+H1	$x \leq 100$	TRUE	PASS	0.380	0.068	0.447	44.9×
s_repeat_50	BASIC+H2	$x \leq 100$ (with offset)	TRUE	PASS	0.452	0.084	0.536	37.4×
s_repeat_100	BASIC+H1	$x \leq 150$ (with offset)	TRUE	PASS	0.382	0.073	0.456	44.0×
s_repeat_50	BASIC+H1	$x \leq 25$	FALSE	PASS	0.413	0.077	0.491	40.9×
s_repeat_100	BASIC+H2	$x \leq 50$	FALSE	PASS	0.389	0.092	0.480	41.8×
s_repeat_50	NONE+H1	$x \leq 50$ (with offset)	FALSE	PASS	0.050	0.104	0.154	1.0×
s_repeat_100	BASIC+H1	$x \leq 100$ (with offset)	FALSE	PASS	0.376	0.096	0.472	42.5×
Specific mode – Nesting Depth								
s_nest_3	NONE	$a, b, c \geq 0$	TRUE	PASS	0.266	0.081	0.347	1.0×
s_nest_4	NONE	$a, b, c, d \geq 0$	TRUE	PASS	0.455	0.161	0.616	1.0×
s_nest_3	BASIC	$c \leq 125$	TRUE	PASS	0.946	0.097	1.043	19.4×
s_nest_4	BASIC+H1	$d \leq 256$	TRUE	PASS	1.729	0.159	1.887	10.8×
s_nest_3	BASIC	$c \leq 135$ (with offset)	TRUE	PASS	0.917	0.127	1.044	19.4×
s_nest_4	BASIC	$d \leq 266$ (with offset)	TRUE	PASS	1.633	0.155	1.788	11.4×
s_nest_3	BASIC+H1	$c \leq 100$	FALSE	PASS	0.940	0.110	1.049	19.3×

Program	Config	Property	Verdict	Result	Build (s)	Solve (s)	Total (s)	Speedup
s_nest_4	BASIC	$d \leq 200$	FALSE	PASS	1.713	0.149	1.862	11.0×
s_nest_3	BASIC+H1	$c \leq 125$ (with offset)	FALSE	PASS	0.922	0.118	1.039	19.5×
s_nest_4	BASIC	$d \leq 256$ (with offset)	FALSE	PASS	1.704	0.161	1.865	11.0×
Specific mode – Wide State								
s_wide	NONE	$a, \ldots, h > 0$	TRUE	PASS	0.204	0.097	0.301	1.0×
s_wide	NONE+H1	$a, \ldots, h > 0$	TRUE	PASS	0.206	0.080	0.286	1.1×
s_wide	BASIC	$a \leq 6$	TRUE	PASS	3.027	0.067	3.094	6.5×
s_wide	BASIC+H1	$a \leq 4$	FALSE	PASS	2.954	0.075	3.029	6.7×
s_wide	BASIC	$h \geq 0$	TRUE	PASS	2.928	0.147	3.075	6.6×
s_wide	BASIC+H1	$h \leq 2211$	TRUE	PASS	2.923	0.085	3.008	6.7×
s_wide	BASIC+H1	$h \leq 2000$	FALSE	PASS	3.269	0.073	3.342	6.1×
s_wide	BASIC+H2	$h \leq 400$	FALSE	PASS	3.194	0.134	3.328	6.1×
s_wide	BASIC+H1	$h \leq 495$	TRUE	PASS	2.976	0.092	3.068	6.6×
Specific mode – Branching Density								
s_branches	NONE	$x \geq 0$	TRUE	PASS	0.441	0.080	0.521	1.0×
s_branches	NONE+H1	$acc \geq 0$	TRUE	PASS	0.431	0.084	0.515	1.0×
s_branches	BASIC+H2	$x \leq 8$	TRUE	PASS	0.480	0.139	0.619	33.0×
s_branches	BASIC	$x \leq 4$	FALSE	PASS	0.531	0.228	0.759	27.0×
s_branches	BASIC	$acc \leq 10$	FALSE	PASS	0.550	0.711	1.260	16.3×
s_branches	NONE	$acc \leq 18$	-	TIMEOUT	0.460	20.014	**T/O**	–
Specific mode – Trigonometric Motion								
s_trig_10	NONE+H1	$ycor \geq 0$	FALSE	PASS	0.134	0.416	0.550	1.7×
s_trig_20	NONE+H1	$ycor \geq 0$	FALSE	PASS	0.132	0.746	0.877	1.8×
s_trig_10	NONE+H1	$\neg pendown$	TRUE	PASS	0.118	0.082	0.201	1.1×
s_trig_20	NONE+H1	$\neg pendown$	TRUE	PASS	0.134	0.086	0.220	1.0×
s_trig_10	NONE+H1	heading $\in \{0, 90, 180, 270\}$	TRUE	PASS	0.169	0.364	0.532	37.8×
s_trig_20	NONE+H1	heading $\in \{0, 90, 180, 270\}$	TRUE	PASS	0.123	0.255	0.378	1.2×
Specific mode – Property Complexity								
s_repeat_100	NONE	single atom ($x \geq 0$)	TRUE	PASS	0.057	0.051	0.107	1.0×
s_repeat_100	BASIC	two-atom conj ($x \geq 0 \wedge x \leq 100$)	TRUE	PASS	0.397	0.077	0.474	42.3×
s_repeat_100	BASIC+H1	101-way exact disjunction on x	TRUE	PASS	0.345	0.195	0.540	37.3×
s_repeat_100	BASIC	$x \leq 100$ (shifted, init= 50)	FALSE	PASS	0.377	0.085	0.462	43.4×
s_repeat_100	BASIC+H1	$x \geq 0 \wedge x \leq 150$ (shifted)	TRUE	PASS	0.368	0.086	0.454	44.2×

Aggregate Results by Mode and Configuration

Tables 9, 10, and 11 present aggregate results for each verification mode across all six configurations. Avg solve time excludes timed-out rows. The best completion rate per table is **bolded**; the **Best** row reports the per-test-case optimum: for each test case the best result across all six configurations is taken, then aggregated.

Table 9: Aggregate results: **Default** mode (38 test cases).

Config	Passed	Failed	Skipped	Completion	Avg Build (s)	Avg Solve (s)
NONE	24	0	14	63%	0.145	1.581
NONE+H1	25	0	13	66%	0.139	2.473
NONE+H2	24	0	14	63%	0.142	2.743
BASIC	33	0	5	87%	1.452	0.713
BASIC+H1	32	0	6	84%	1.482	0.415
BASIC+H2	34	0	4	**89%**	1.481	0.674
Best	**37**	**0**	**1**	**97%**	**0.525**	**0.119**

Table 10: Aggregate results: **Universal** mode (38 test cases).

Config	Passed	Failed	Skipped	Completion	Avg Build (s)	Avg Solve (s)
NONE	28	0	10	74%	0.112	1.310
NONE+H1	28	0	10	74%	0.110	1.527
NONE+H2	30	0	8	79%	0.113	1.565
BASIC	38	0	0	**100%**	1.163	0.061
BASIC+H1	38	0	0	**100%**	1.143	0.061
BASIC+H2	38	0	0	**100%**	1.147	0.078
Best	**38**	**0**	**0**	**100%**	**0.323**	**0.093**

Table 11: Aggregate results: **Specific** mode (52 test cases).

Config	Passed	Failed	Skipped	Completion	Avg Build (s)	Avg Solve (s)
NONE	19	0	33	37%	0.221	0.256
NONE+H1	20	0	32	38%	0.219	0.200
NONE+H2	20	0	32	38%	0.218	0.685
BASIC	48	0	4	**92%**	1.793	0.474
BASIC+H1	48	0	4	**92%**	1.779	0.533
BASIC+H2	44	0	8	85%	1.780	0.175
Best	**51**	**0**	**1**	**98%**	**0.903**	**0.148**

How to Run

The correctness suite is driven by `pytest` from the `CHC-Verification/correctness_tests/` directory. Parallel execution via `pytest-xdist` (`-n auto`) is recommended.

```
# runs all tests in parallel
pytest -n auto -q

# one module only
pytest -n auto -q <test_module.py>
```

Performance Analysis

This section analyses the timing data collected across the performance tests in the previous section. All timings exclude timed-out rows (20 s limit) when computing averages.

Build Time Analysis Across Verification Modes

Figure 5 plots average build times for each configuration across the three verification modes.

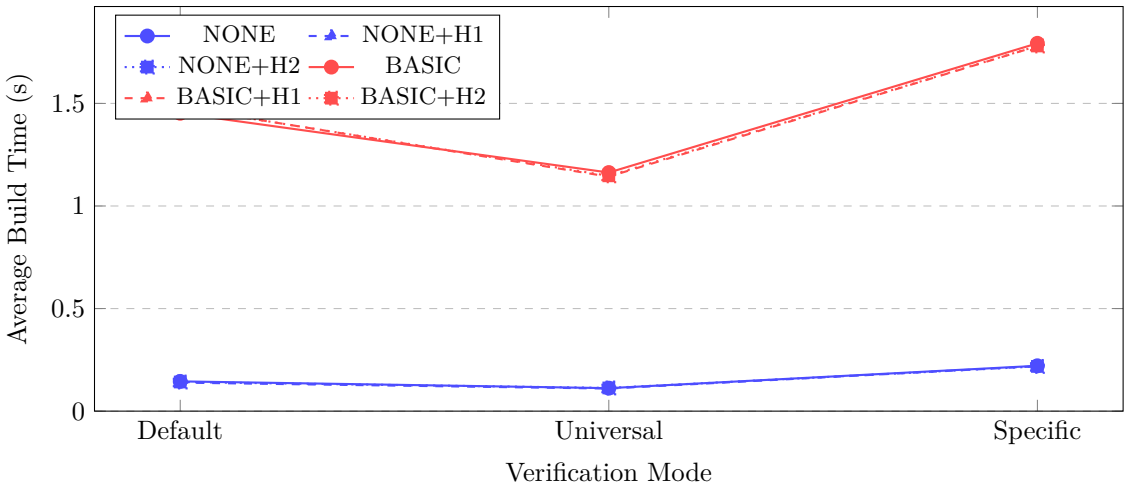

Figure 5: Average build time per configuration across verification modes. Blue lines = NONE family (no loop optimisation); red lines = BASIC family (with loop-collapse). Hints do not materially affect build time.

The NONE family stays flat and cheap across all modes (0.11–0.22 s), because the CHC system is built with no additional pre-processing. BASIC-family configurations pay a fixed overhead of roughly 1.1–1.8 s for the loop-collapse pass; this cost is mode-independent, rising only with program size. Within BASIC, Specific mode is the most expensive (1.79 s) because its programs carry additional symbolic parameters that expand the loop-collapse computation. Hints have negligible effect on build time in both families.

Solve Time Analysis Across Verification Modes

Figure 6 plots average solve times (excluding timeouts).

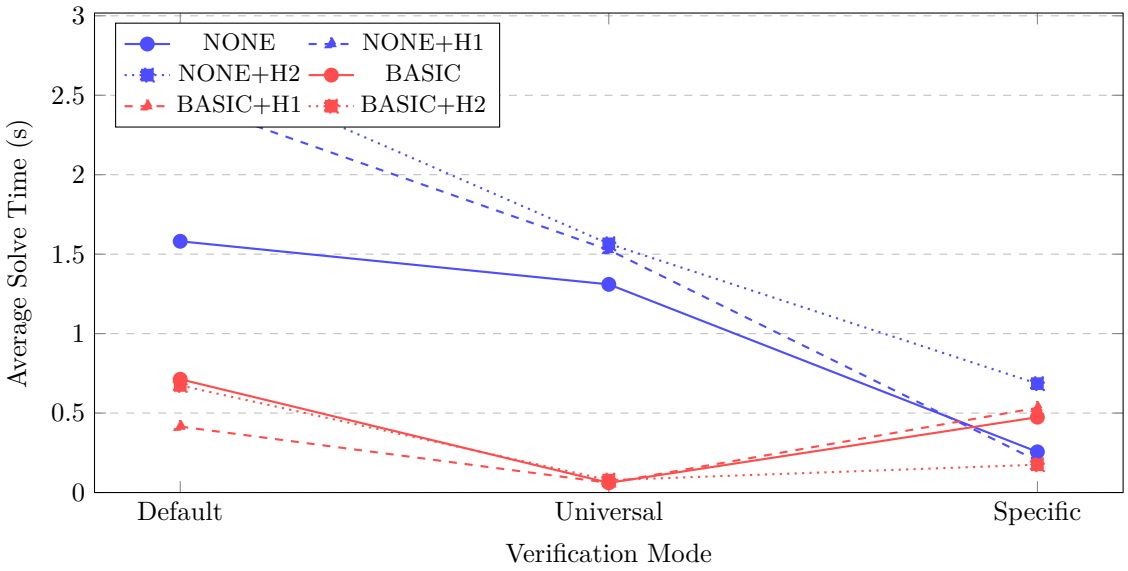

Figure 6: Average solve time per configuration (timeouts excluded from average). H2 (`heading_on_grid_always`) can *raise* solve time under NONE because the added assumption enlarges the CHC formula without the structural simplification that BASIC provides.

The key contrast is between NONE and BASIC is, BASIC reduces the number of CHC rules seen by Spacer, so the solver's *per-query* time drops dramatically, especially for Universal mode (from ≈ 1.3 s down to 0.06 s). For NONE-family configurations, H2 increases average solve time in Default mode ($1.58 \rightarrow 2.74$ s) because

injecting the heading constraint adds a large disjunctive guard to every rule, and without loop-collapse Spacer must reason over many more rule instances. Under BASIC the situation reverses: the collapsed loop already carries the grid invariant so H2's guard is cheap to discharge, dropping Specific solve time from 0.47 to 0.18 s.

Timeout and Optimisation Benefits

Verdict distribution. Figure 7 shows the distribution of outcomes (proved/UNSAT, counterexample found/SAT, timeout) for all six configurations across the three modes.

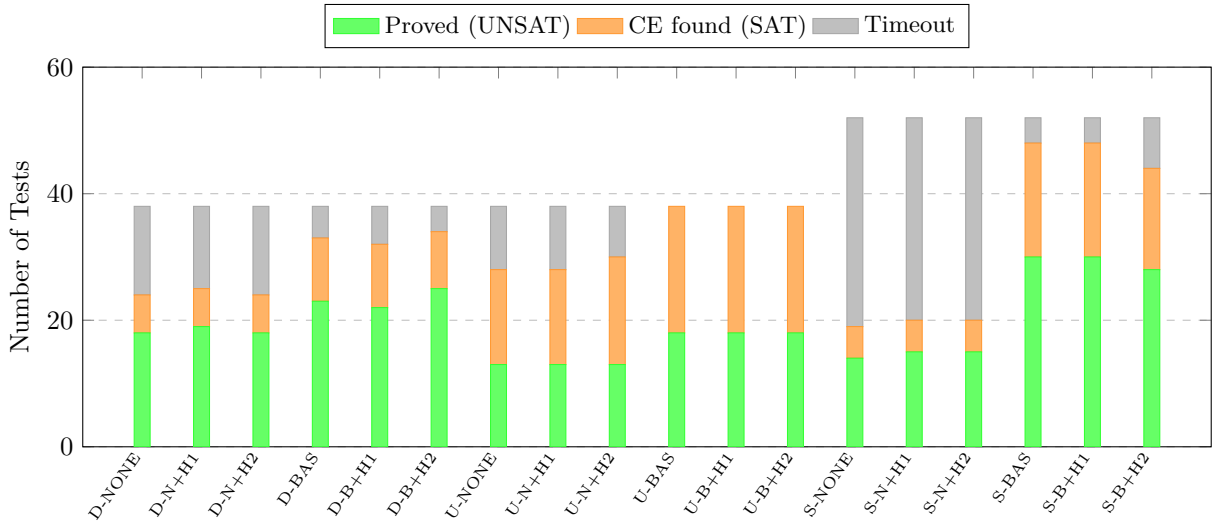

Figure 7: Verdict distribution across all modes (D=Default, U=Universal, S=Specific) and configurations (N=NONE, BAS=BASIC). Green = invariant proved; orange = expected counterexample found; gray = solver timed out.

Incremental benefit of the loop-collapse optimisation (BASIC). Table 12 shows how many previously-timed-out tests become solvable when NONE is upgraded to BASIC, and how many (if any) regress.

Table 12: Tests rescued from timeout by the BASIC loop-collapse optimisation (NONE → BASIC). Regressions are tests that timed out under BASIC but not under NONE.

Mode	Rescued	Regressions	Net gain
Default	13	4	+9
Universal	10	0	+10
Specific	31	2	+29
Total	**54**	**6**	**+48**

Milestone 2 vs. Milestone 3 comparison. Table 13 compares test completion rates between Milestone 2 and Milestone 3. Despite the three-fold reduction in time budget, M3 solves substantially more tests in every mode and configuration.

Table 13: Test completion rates: Milestone 2 (20 s timeout) vs. Milestone 3 (20 s timeout).

	Default (38 tests)		Universal (38 tests)		Specific (52 tests)	
Config	**M2**	**M3**	**M2**	**M3**	**M2**	**M3**
NONE	39%	63%	68%	74%	25%	37%
BASIC	58%	87%	89%	100%	35%	92%
BASIC+hint	61%	89%	92%	100%	37%	92%

The biggest improvement is in specific mode: *Milestone-2* `BASIC` solved only 18 of 52 tests (35%), while *Milestone-3* `BASIC` solves 48 of 52 (92%). The improvement comes from the loop-collapse encoding, which transforms deep parametric loops (e.g. `s_repeat_100`, `s_nest_4`, `s_wide`) into compact fixed-point systems that SPACER closes in under 0.2 s.

A secondary improvement is build time for Universal-mode trigonometric programs: *Milestone-2* `BASIC` built `perf_trig_10` and related programs in 190–210 s each due to symbolic expansion of `repeat`/`forward` loops; M3 BASIC handles the same programs in ≈ 1.7 s (a $\approx 120\times$ reduction). In exchange, *Milestone-3* `BASIC` spends 3–9$\times$ more build time than M2 on simple loop programs (e.g. `repeat_10`) because the new pass performs a deeper algebraic analysis.

Hint benefits (H1 and H2). Table 14 shows the number of timeouts eliminated by each hint independently and the number of tests that *regressed* (a previously-passing configuration now times out) under the assumption hint H2.

Table 14: Timeout reduction by hints relative to the no-hint baseline. "Rescued" = timeout→solved; "Regressions" = solved→timeout.

Transition	Mode	Rescued	Regressions
NONE → NONE+H1	Default	1	0
	Universal	0	0
	Specific	1	0
NONE → NONE+H2	Default	0	0
	Universal	2	0
	Specific	1	0
BASIC → BASIC+H1	Default	0	1
	Universal	0	0
	Specific	0	0
BASIC → BASIC+H2	Default	3	2
	Universal	0	0
	Specific	0	4

Representative rescued tests. Table 15 lists specific program–property pairs that time out under the baseline and are rescued by a particular configuration. These cases demonstrate that the choice of optimisation and hint is *necessary*.

Table 15: Specific tests that time out (≥ 20 s) under the baseline and are rescued by the indicated configuration. Solve time shown is for the rescuing configuration.

Program	Property	Mode	Rescuing config	Solve (s)
Tight upper bounds — require BASIC loop-collapse				
`repeat_100`	$x \leq 100$	Default	BASIC	0.034
`repeat_50`	$x \leq 50$	Default	BASIC	0.052
`deep_nest_3`	$c \leq 64$	Default	BASIC	0.069
`deep_nest_4`	$d \leq 81$	Default	BASIC	0.078
`many_branches`	$x \leq 6$	Default	BASIC	0.092
`wide_vars`	$a \leq 6$	Default	BASIC	0.078
`s_repeat_100`	$x \leq 100$	Specific	BASIC	0.080
`s_nest_4`	$d \leq 256$	Specific	BASIC	0.150
`s_wide`	$a \leq 6$	Specific	BASIC	0.067
Heading-grid properties — require BASIC+H2				
`turns_20`	heading on 24-grid	Default	BASIC+H2	0.079
`turns_20`	heading $\in \{0, 345\}$	Default	BASIC+H2	0.318
`turns_50`	heading $\in \{0, 345\}$	Default	BASIC+H2	0.287
Property complexity — require NONE+H1				
`repeat_10`	11-way exact disj.	Default	NONE+H1	19.756

Overall Performance

Figure 8 breaks completion rate down per mode, allowing comparison across the three verification settings.

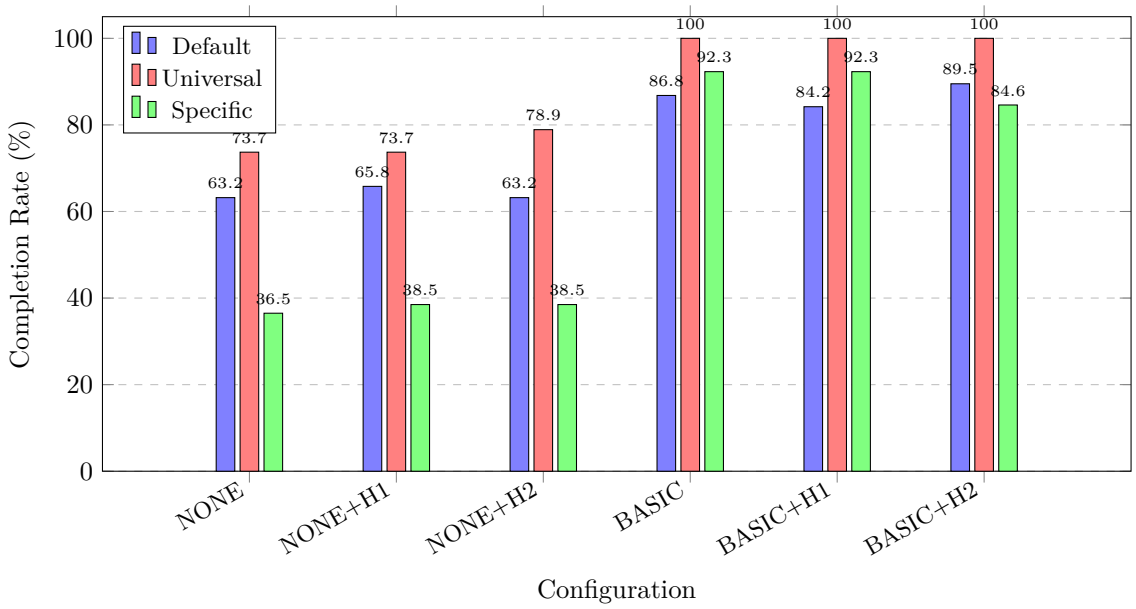

Figure 8: Completion rates broken down by verification mode. Universal mode reaches 100% under all BASIC variants. Specific mode is most sensitive to hint choice: H2 introduces regressions under BASIC.

When to Use Each Configuration

- **Prefer `BASIC` over `NONE`.** The M3 loop-collapse optimisation rescues 48 net tests from timeout compared to NONE, and raises the overall completion rate from 55% to 93%. The only cost is a fixed build overhead ($\approx$ 1.1–1.8 s) that is negligible.

- **Add `H1` for trig-heavy programs with heading constraints.** H1 (`check_heading_always_on_grid`) provides a moderate benefit for programs that turn on a 15-degree grid by adding a lightweight query-time constraint that helps Spacer close the heading-invariant without extra CHC rules.
- **Use `H2` under `BASIC`.** H2 (`heading_on_grid_always`) is an *assumption* hint: it strengthens the initial state, which is only sound if the program genuinely keeps heading on the grid throughout. Under `NONE`, it enlarges every CHC rule with a disjunctive guard, worsening solve time. Under `BASIC`, it rescues 3 Default tests and reduces Specific solve time from 0.47 to 0.18 s.
- **Tight-bound properties benefit most from `BASIC`.** Properties such as $x \leq N$ (exact upper bound) and deep nesting invariants require Spacer to discover non-trivial loop invariants. Under `NONE`, every such test times out; under `BASIC`, the collapsed loop exposes the invariant structure directly to Spacer, reducing solve time from timeout to < 0.1 s.

Future Work

Future work to make the verifier stronger and easier to use includes:

1. **ML-guided verification.** Many results become `UNKNOWN` because hints and solver settings are fixed. A useful next step is to use lightweight ML models over IR features (loop depth, branching, variable count, and property form) to choose hints, timeout values, and optimization levels automatically. This can reduce failed runs and improve solver performance on different program families.
2. **Generalize beyond the current heading-grid discipline.** The current verifier assumes heading values stay on a 15-degree grid. If this cannot be proved, verification often returns `UNKNOWN`. We plan to extend support to arbitrary angles by adding stronger handling of non-linear trigonometric updates, using sound abstractions, piecewise-linear approximations, or non-linear solver backends.
3. **A portfolio of multiple SMT solvers.** The current pipeline depends on one Horn-clause solving stack. A solver portfolio can run multiple backends with fallback per property and return the strongest result available (`PASSED`/`FAILED` preferred over `UNKNOWN`) under a shared timeout policy.
4. **Extra properties and richer user semantics.** Basic arithmetic, geometry, pen, and heading helpers already exist. Next, we should add higher-level property types such as temporal properties (`always(phi)`, `eventually(phi)`, `until(phi, psi)`), bounded-step goals (`within_k_steps(phi, k)`), monotonicity checks (`monotonic_nondecreasing(v)`), and richer geometric regions (such as polygon containment). This would let users express trace-level behavior, not just state-level constraints, while still compiling safely to CHC formulas.
5. **Counterexamples that teach.** Failed queries currently return solver-level outputs that are hard to read. Future versions should rebuild source-level traces, mark the first instruction where the property breaks, and show a minimal witness over only relevant variables. This would make failures easier to debug and learn from.
6. **Scalability.** Universal mode and deep control flow increase relation size and rule count, which hurts runtime and raises `UNKNOWN` rates. A key direction is parallelism: multi-threaded checking of independent property batches, concurrent solver portfolios with shared timeout budgets, and parallel preprocessing (IR analysis, slicing, and summarization). Along with compositional summaries and incremental caching, this can improve throughput and reduce wall-clock time on multi-core machines.
7. **Optimisations.** Beyond current basic optimizations, we plan the following sound performance improvements:
   - **Property-impact slicing:** keep only instructions and variables that can affect a given property, and generate smaller CHC systems.
   - **Adaptive timeout scheduling:** assign per-property timeout budgets from simple complexity signals instead of one fixed timeout.
   - **Parallel property queues:** solve independent properties on worker threads and merge results in deterministic API order.
   - **Incremental lemma reuse:** reuse learned invariants across scopes (`all`, `terminating`, `at_pc`, `upto_pc`) for the same IR.
   - **Counterexample-guided refinement:** when a query is `UNKNOWN`, refine only ambiguous transitions instead of strengthening the full model.

Source Code

The complete implementation is hosted at: `https://github.com/chiron-verification/milestone-2`

- **Milestone-1 commit:** `0f99ade6d4aa2516f9e47e013ad06d29e8b8a2f7`
- **Milestone-2 commit:** `4d3e3e6499b5eea03d25f0b0a3e1bf7a4dcd998b`
- **Milestone-3 (Final) commit:**

The correctness test suite (`correctness_tests/`) has substantially expanded across `test_default.py`, `test_specific.py`, `test_universal.py`, and `test_semantics.py` along with many new `.tl` fixtures and shared helpers to improve coverage of arithmetic, control-flow, and heading-related properties.

In addition, a dedicated performance testing setup (`performance_tests/`) is added with mode-wise test runners, benchmark fixtures, and reusable harness/configuration files to measure solver behavior on deep nesting, wide-state, branch-heavy, and high-iteration workloads.

Running the Verifier

Prerequisites and Setup

The verifier requires Python 3.11 or later, Z3 4.13 or later (installed as the `z3-solver` PyPI package), and ANTLR 4 (available as `antlr4-python3-runtime`).

```
pip install z3-solver antlr4-python3-runtime
```

Python API

Entry Point: `CHC_Verification`

File: CHC-Verification/safety_properties.py

```
from safety_properties import CHC_Verification
from variable_name_detection_in_IR import OptimizationLevel

result = CHC_Verification(
    file_name,
    mode,
    user_properties,
    params            = None,
    input_ranges      = None,
    property_scope    = "all",
    pc_target         = None,
    hints             = ["check_heading_always_on_grid",
                         "check_termination"],
    timeout_ms        = 60_000,
    optimization_level= OptimizationLevel.NONE,
)
```

Parameters.

Parameter	Type	Description
`file_name`	`str`	Path to the Chiron `.tl` source file to verify.
`mode`	`str`	Verification mode. One of `"default"`, `"specific"`, or `"universal"`.
`user_properties`	`list`	List of property objects. Each object must expose a `.name` (string label) and `.expr` (Z3-style Python expression string). Variable names match program source without the leading `:` (e.g. `"x >= 0"` for Chiron variable `:x`).
`params`	`str | None`	Required only in `"specific"` mode. A string representation of a Python dictionary mapping colon-prefixed variable names to numeric values. Example: `"{':x': 10, ':y': 5}"`. Omitted variables default to zero.

Parameter	Type	Description
input_ranges	dict | None	Used only in "universal" mode to restrict user variables. Keys are bare variable names (no :). Values may be an int (fixed constant), a (lo, hi) integer interval (inclusive), or None (unconstrained). Unknown variable names are rejected.
property_scope	str	One of "all" (default), "terminating", "at_pc", or "upto_pc". Controls which reachable states the properties are checked against.
pc_target	int | None	Required for "at_pc" and "upto_pc" scopes; must be an integer between 0 and the terminal PC (inclusive). Forbidden for "all" and "terminating" scopes.
hints	list[str]	Solver behaviour hints. Supplying both members of a mutually exclusive pair (prove vs. assume) is rejected.
timeout_ms	int	Per-query solver timeout in milliseconds. Default: 60_000 (60 seconds). When a query times out the corresponding property is recorded as UNKNOWN.
optimization_level	OptimizationLevel	OptimizationLevel.NONE (default) uses the standard per-instruction encoding. OptimizationLevel.BASIC enables the turn-safety and loop-summarization pipeline. Incompatible with property_scope="at_pc" or "upto_pc".

Property objects.

Any object with a string .name and a string .expr field is accepted; no specific class is required. The expression in .expr is evaluated in a context that provides xcor, ycor, heading, pendown, And, Or, Not, and every user variable and loop counter by their bare name (without the : prefix used in Chiron source).

```
class UserProperty:
    def __init__(self, name: str, expr: str):
        self.name = name
        self.expr = expr
```

ReturnValue.

Field	Type	Description
.status	str	Overall verdict: "PASSED", "FAILED", or "UNKNOWN".
.heading_grid_safe	str	Heading grid check outcome: "PASSED", "FAILED", or "UNKNOWN".
.error	ReturnError	ReturnError.SUCCESS, .ERROR (invalid input), or .UNKNOWN.
.build_time	float	Wall-clock time (seconds) to parse and build the CHC encoding.
.solve_times	list[float]	Per-property solve time in seconds, in property list order.
.passing_properties	list	[name, invariant] pairs for each PASSED property.
.failing_properties	list	[name, counterexample] pairs for each FAILED property.
.unknown_properties	list[str]	Names of all UNKNOWN properties.
.expr	str	Human-readable summary string.
.advice	str	Human-readable suggestion for next steps.

Semantic Helper API: CHC_Verification_semantic

File: CHC-Verification/semantic_properties.py

CHC_Verification_semantic wraps the core verifier with a transpilation step that converts readable helper-style property strings into raw Z3-compatible expressions before they reach eval().

```
from semantic_properties import CHC_Verification_semantic

result = CHC_Verification_semantic(
    file_name=...,
    mode=...,
    user_properties=...,  # helper-style expressions accepted
    # all other kwargs forwarded to CHC_Verification
)
```

Property strings may use any of the helper functions listed below in Table 18; raw Z3-style expressions (`x >= 0`, `And(...)`) are also accepted unchanged.

Helper	Args	Expansion
`is_nonnegative(v)`	1	`v >= 0`
`is_positive(v)`	1	`v > 0`
`is_negative(v)`	1	`v < 0`
`is_between(v, lo, hi)`	3	`And(v >= lo, v <= hi)`
`is_strictly_between(v, lo, hi)`	3	`And(v > lo, v < hi)`
`is_one_of(v, a, b, ...)`	≥ 2	`Or(v==a, v==b, ...)`
`is_multiple_of(v, n)`	2	`v % n == 0`
`sum_is(a, b, c)`	3	`a + b == c`
`pen_is_up()`	0	`Not(pendown)`
`pen_is_down()`	0	`pendown`
`x_in_range(lo, hi)`	2	`And(xcor >= lo, xcor <= hi)`
`y_in_range(lo, hi)`	2	`And(ycor >= lo, ycor <= hi)`
`position_in_box(x0,x1,y0,y1)`	4	`And(xcor>=x0, xcor<=x1, ycor>=y0, ycor<=y1)`
`at_position(x, y)`	2	`And(xcor==x, ycor==y)`
`on_diagonal()`	0	`xcor == ycor`
`heading_is(h)`	1	`heading == h`
`heading_cardinal()`	0	`Or(heading==0, 90, 180, 270)`
`heading_on_15_grid()`	0	`heading % 15 == 0`
`var_bounded(v, lo, hi)`	3	`And(v >= lo, v <= hi)`
`vars_all_nonnegative(a,b,...)`	≥ 1	`And(a>=0, b>=0, ...)`
`implies(p, q)`	2	`Or(Not(p), q)`
`relation_guarded(cond, p, q)`	3	`Or(And(cond,p), And(Not(cond),q))`
`all_of(p, q, ...)`	≥ 1	`And(p, q, ...)`
`any_of(p, q, ...)`	≥ 1	`Or(p, q, ...)`
`not_prop(p)`	1	`Not(p)`

Table 18: Selected semantic helper functions available in `CHC_Verification_semantic`.

CLI Interface

The CHC verifier is integrated into `chiron.py` and is invoked by adding `-chc` to any standard `chiron.py` command line.

```
cd Chiron-Framework/ChironCore
python chiron.py -chc [options] <program.tl>
```

CHC-specific flags.

Flag	Default	Description
`-chc`	—	Enable CHC verification. Must be present to activate any of the flags below.
`-chc_mode`	`default`	Verification mode. Choices: `default`, `specific`, `universal`.

Flag	Default	Description
`-chc_props`	*(none)*	Properties to verify, as a semicolon-separated list of `name:expr` pairs. `name` is a label; `expr` may be a raw Z3-style expression (e.g. `x >= 0`) *or* a semantic helper call (e.g. `is_nonnegative(x)`, `position_in_box(-100,100,-100,100)`); both are accepted and helper calls are transpiled automatically. Variable names omit the `:` prefix. Example: `"x_pos:is_nonnegative(x);y_bound:y <= 100"`. Omitting this flag runs the heading-grid check only.
`-chc_scope`	`all`	Property scope. Choices: `all`, `terminating`, `at_pc`, `upto_pc`.
`-chc_pc`	*(none)*	Target PC (integer) for `at_pc` or `upto_pc` scope.
`-chc_timeout`	`60000`	Solver timeout per query in milliseconds.
`-chc_hints`	*default hint*	Comma-separated solver hints.
`-chc_opt`	`NONE`	Encoding optimization level. Choices: `NONE`, `BASIC`. `BASIC` incompatible with `at_pc`/`upto_pc` scope.
`-d` / `-params`	*(none)*	Initial variable values for `specific` mode, as a Python dict string. Example: `-d '{":x": 10, ":y": 5}'`.

CLI Output

After the solver completes, `chiron.py` prints a structured result block:

```
=== CHC Verification Result ===
Status          : PASSED
Heading-grid    : PASSED
Build time      : 0.031s
Passing         : ['x_nonneg']
Message         : All properties satisfied.
Advice          : The program is safe to run with respect to the specified properties.
```

AI Use in the Project

In this project, we utilized AI tools to enhance our workflow and improve the quality of our work.

1. **Are you using AI for coding your implementation?** Yes, we used AI tools in this project.
2. **If yes, what models are you using?** We used models like OpenAI's GPT-5.3 Codex, GPT-5.4 Codex, Claude Code and GitHub Copilot.
3. **If using AI, how are you using AI and what percentage (approx) of your code is AI generated?** We used AI primarily for generating boilerplate code, suggesting improvements, and debugging. Approximately $40\% - 50\%$ of our code was generated or assisted by AI tools.
   - AI was primarily used for generating boilerplate code and usage of correct syntax, which was then reviewed and modified by us to fit our specific requirements.
   - We also used AI for testing, where it helped in generating test cases and suggesting improvements to our existing tests.
   - Additionally, AI was used for the generation of documentation and figures in the report, where it assisted in summarizing the directory structure and usage guide, as well as making the figures in the report with careful and structured instructions about the layout.

Declaration

1. We declare that the work presented in this report is our own and has been carried out in accordance with the guidelines set out by the course.
2. We have not copied any part of this report from any other source, and we have not submitted this report to any other course or institution for assessment.
3. We understand that any violation of this declaration may result in disciplinary action.